

Orrery

TECHNICAL OVERVIEW

Inside Orrery

How code becomes motion: the language, the solver, the renderer, its dependencies, strengths and limits

Version 1.0.0 · language 1
Sivaprasad Murali and the Orrery contributors · 1 November 2026
Apache License 2.0

Contents

	Section
1	What Orrery is
2	The pieces and how they fit
3	From code to picture: the pipeline
4	Reading the code: lexer and parser
5	Building the model: the compiler
6	Solving a moment in time
7	Closed loops: the linkage solver
8	Drawing it: the preview renderer
9	Errors, warnings and the overlap check
10	Sound
11	Exports, sharing and the web app
12	Tools around the language
13	Dependencies
14	Strengths
15	Weaknesses and limits
16	Quality: how it is tested
17	Key numbers and glossary

In one paragraph

Orrery is a small language and a browser studio for animating machines. You write what the parts are, how they connect (hinges, sliders, gears, belts, pins) and which few parts are driven. A compiler turns that into a model, a kinematic solver works out where every part is at any instant, and three.js draws it. The same model is exported as video with generated sound, GIF, PNG frames or a self-contained 3D player, and is readable by people, command-line tools and AI assistants alike.

SECTION 1

What Orrery is

Most animation tools ask you to move every part yourself: keyframes in a timeline, or numbers you compute in code. Machines do not work like that. A crank turns, and the connecting rod and piston follow because they are connected. Orrery lets you write exactly that, and nothing more: **joints, not keyframes**.

A project answers three questions, each in its own file, much as HTML, CSS and animation rules divide a web page:

File	Answers	Holds
`.scene`	What exists?	Parts (38 shape types), groups, anchors, joints and connections, the camera, lights, sounds, scene settings
`.style`	How does it look?	CSS-like selectors with colour, material, roughness, metalness, outline, opacity, luminosity, labels, trails
`.motion`	How does it move?	Time blocks (forever, 0s..2s, then) that drive parts, joints and the camera, change colour, light and opacity, and play sounds

A four-bar linkage: one driven part, three that follow

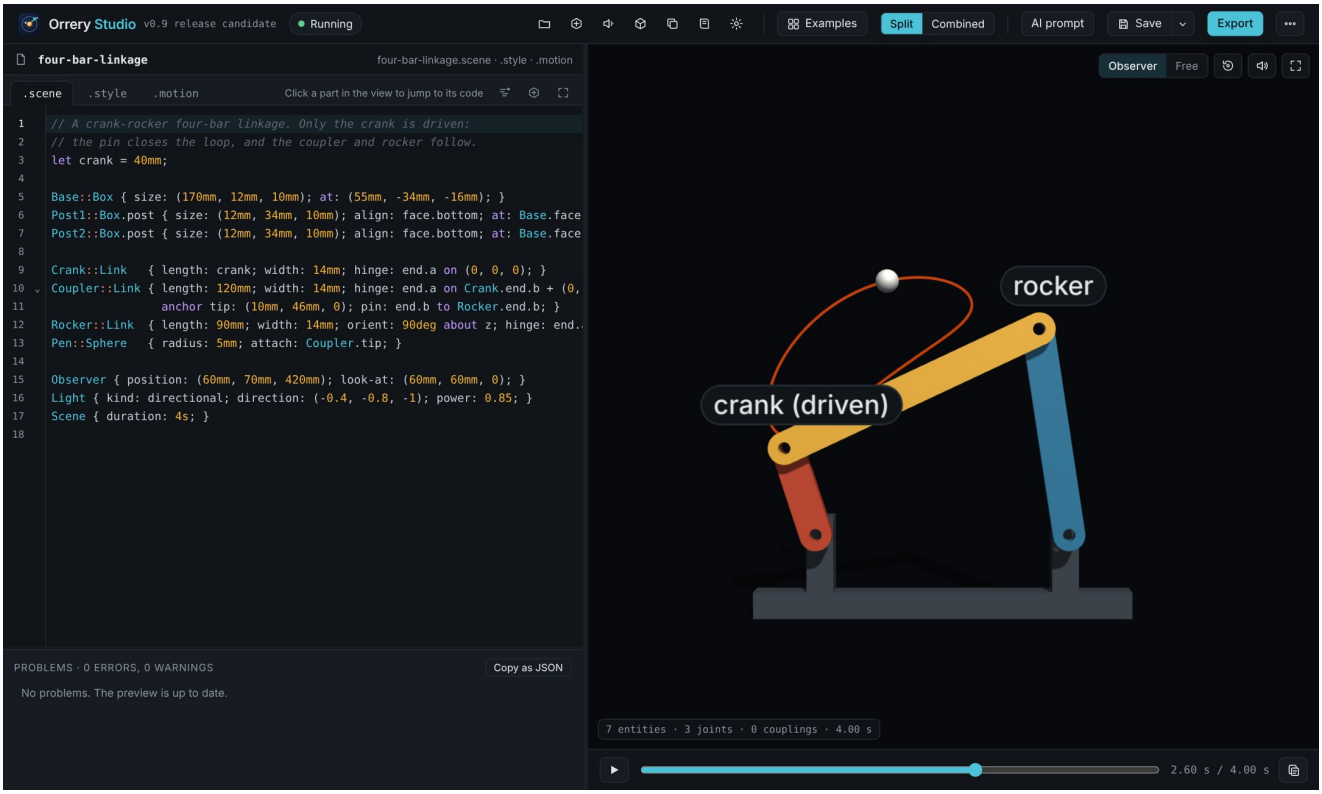
```
@orrery 1

@scene
Crank::Link   { length: 40mm; width: 14mm; hinge: end.a on (0, 0, 0); }
Coupler::Link { length: 120mm; width: 14mm; hinge: end.a on Crank.end.b + (0, 0, 6mm);
                anchor tip: (10mm, 46mm, 0); pin: end.b to Rocker.end.b; }
Rocker::Link  { length: 90mm; width: 14mm; orient: 90deg about z; hinge: end.a on (110mm, 0, 0); }
Pen::Sphere   { radius: 5mm; attach: Coupler.tip; }

@style
Pen { color: #ffd27a; luminosity: 80%; trail: #ffb84d; }

@motion
Crank { forever: rotate: 180deg/s; }      // the only motion rule
```

Only the crank has a motion rule. The coupler and rocker are hinged, and the pin joins their free ends, so the solver works out their angles at every instant. The pen traces the coupler curve and glows.



The studio: code on the left with live hints and problems; the solved animation on the right, with a timeline to scrub through time.

Who it is for

- **Explainers and teachers** who want an accurate mechanism animation without learning CAD or keyframing.
- **Engineers and makers** who want to show how a design moves, from text they can version and diff.
- **AI assistants and agents**: the syntax is short, errors name their fix, and the exact state at any moment can be read back as text, so a model can check and correct its own work.

SECTION 2

The pieces and how they fit

Orrery ships as static files. The language and the studio live in a single HTML file so that it can be opened from disk, hosted anywhere, installed as an app or downloaded as one offline file. Everything else either reads from that file or serves it.

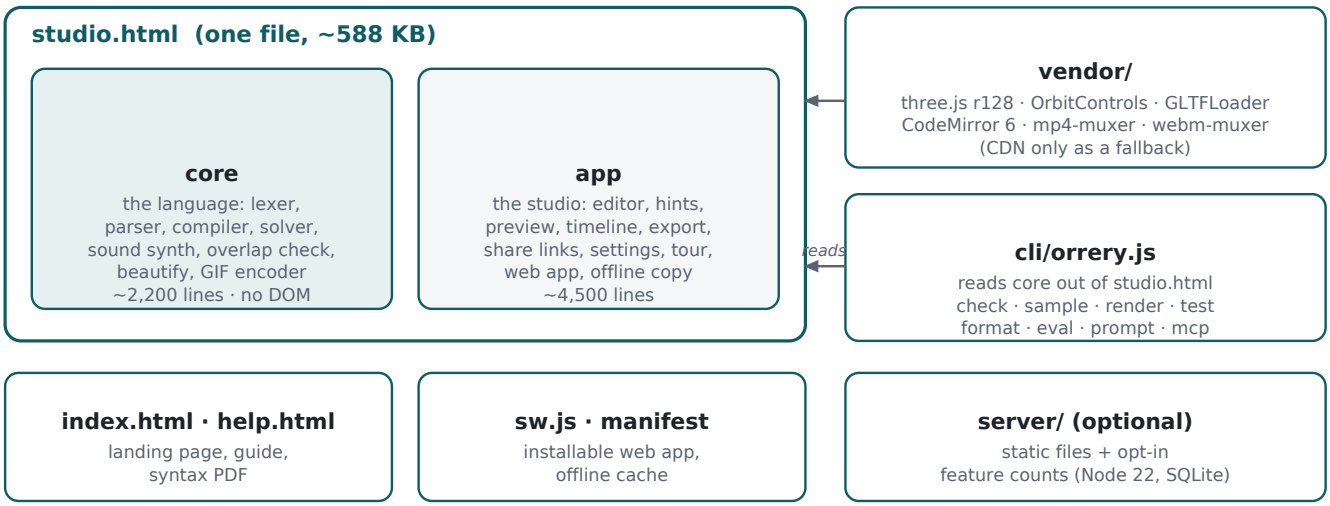


Figure 1: the parts of the project and what depends on what.

Component	Role	Size
core (in studio.html)	The language: lexer, parser, compiler, solver, sound synthesis, overlap check, beautifier, GIF encoder. Uses no browser APIs except three.js maths, so it also runs in Node.	~2,200 lines, 178 KB
app (in studio.html)	The studio interface: CodeMirror editor, type hints, previews, timeline, export, sound panel, share links, settings, tour, web-app features.	~4,500 lines, 347 KB
vendor/	Bundled libraries (section 13). Loaded locally; a public CDN is used only if the files are missing.	~1.2 MB
cli/orrery.js	The command line. It extracts the core from studio.html at start-up, so the browser and the CLI can never disagree about what a project means.	427 lines
index.html, help.html	Landing page and the language guide, with recipes that open in the studio as share links.	
sw.js, manifest.webmanifest	Service worker and manifest: offline use and installation as an app.	
server/ (optional)	Serves the files and counts feature use for visitors who opt in. No accounts.	158 lines
tests/, evals/	Saved motion snapshots for every example, and 12 graded AI tasks with reference answers.	

SECTION 3

From code to picture: the pipeline

The central design choice is a split between work done **once per edit** and work done **once per moment**. Compiling resolves names, builds geometry and prepares every connection. Solving then answers one question, cheaply and repeatedly: given a time t, where is every part? Everything downstream (the preview, exports, the CLI, the AI state text) is a consumer of that one function.

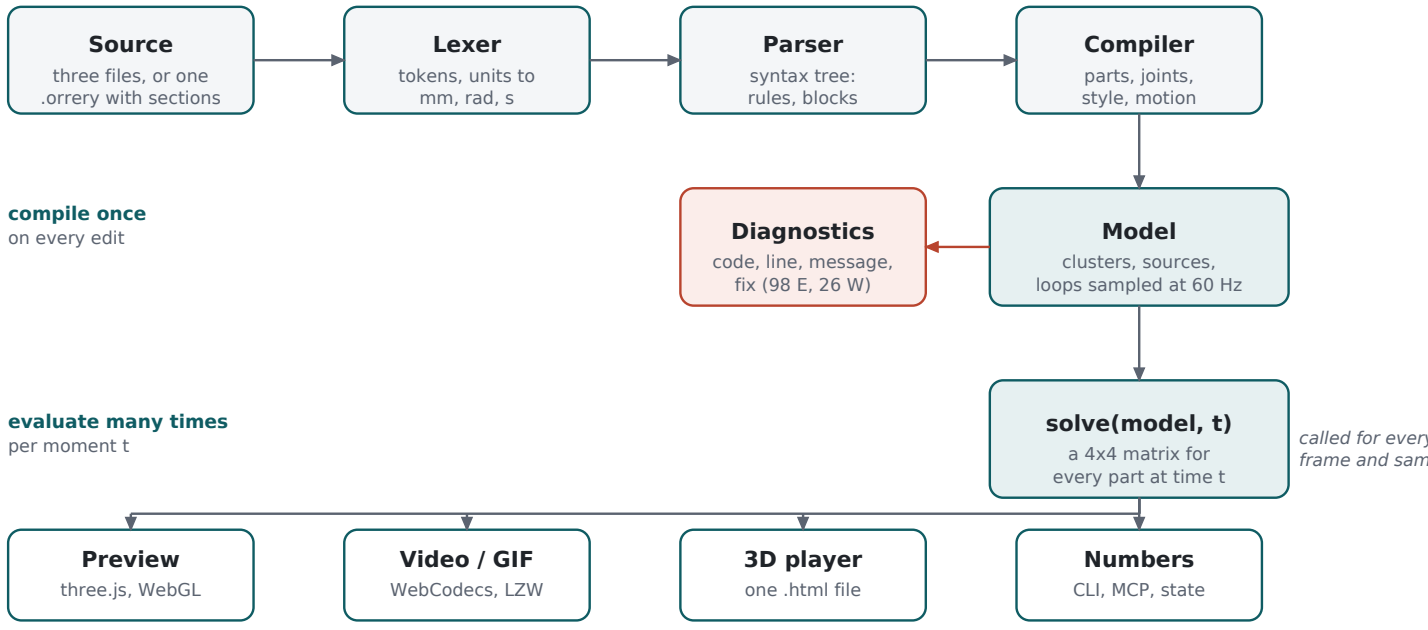


Figure 2: the pipeline. The top row runs on every edit (debounced as you type); solve(model, t) runs for every frame drawn, exported or sampled.

Stage	Input	Output	When
Lex	Text of each file	Tokens with line and column; numbers converted to base units	Each edit

Stage	Input	Output	When
Parse	Tokens	A syntax tree of statements, rules, blocks and properties	Each edit
Compile	Three trees	A model: parts with geometry, clusters, joint and motion sources, styles, sound events, loop samples, diagnostics	Each edit
Overlap check	Model	W0560 warnings for parts that pass through each other	0.7 s after an edit, within a time budget
Solve	Model, time t	A 4x4 world matrix per part	Every frame and sample
Draw / export	Matrices, live style	Pixels, video, GIF, player, numbers	Continuously or on request

SECTION 4

Reading the code: lexer and parser

Files and sections

A project is three files or one `.orrery` file. In the single file, lines `@scene`, `@style` and `@motion` start each section, and an optional first line `@orrery 1` records the language version. The splitter keeps a map from section lines back to file lines, so every problem is reported where you see it. A project written for a newer major version raises W0100.

The lexer

The lexer is a single pass over the text using sticky regular expressions. It skips whitespace and `//` and `/* */` comments, and produces numbers, identifiers, strings, colours and punctuation, each with its line and column.

- **Units are resolved at once.** `40mm`, `4cm` and `0.04m` all become 40 (millimetres); `90deg` becomes pi/2 radians; `500ms` becomes 0.5 seconds; `50%` becomes 0.5. Rates (`/s`) and accelerations (`/s2`) are tagged, which later decides how a value behaves over time.
- **Identifiers may contain hyphens** (`ease-in-out`, `look-at`, `label-color`). The compiler warns (W0105) when `gap-10mm` looks like a subtraction.
- **Errors stop early and say why:** an unknown character, an unterminated comment or string, each with a numbered code and a fix.

The parser

A hand-written recursive-descent parser builds a plain syntax tree. Scene statements are entities (`Name::Type.class { ... }`), groups with members, `define` templates with parameters, `let` constants and settings blocks (Observer, Light, Scene). Expressions support arithmetic, vectors, functions (sin, cos, gradient, cubic-bezier...), anchor paths (`Crank.end.b`) and changes (`0deg -> 90deg ease-in-out`). In motion, a rule holds time blocks; properties belong to the block label before them.

A beautifier works from the same rules: it re-lays the text like CSS (one declaration per line, even indents) but changes only whitespace, which the test suite verifies by compiling every example before and after.

SECTION 5

Building the model: the compiler

`compile(files)` turns the three trees into a model in a fixed order. Each step catches its own mistakes and records them as diagnostics instead of stopping, so one typo does not hide everything else.

Step	What	How
1	Constants and templates	`let` values are substituted everywhere; `define` types are expanded with their parameters (E0130-E0139). `repeat` makes numbered copies.
2	Parts and geometry	Each entity becomes a three.js mesh built from its shape parameters (gears get real involute-style teeth, springs real coils). Groups hold members in their own frame. Each shape publishes named anchors: faces, edges, ends, axes, centre of mass.
3	Placement	`align`, `at` and `orient` place parts relative to anchors of earlier parts, giving every part its rest matrix M0.
4	Connections	`attach` glues parts into clusters. `hinge` and `slider` create joints between clusters. `mesh`, `couple` and `belt` become spin edges with ratios from tooth counts or radii. `pin` records a loop closure.
5	Style	Selectors are matched to parts; materials set metalness and roughness; colours, gradients and alpha are applied; luminosity, labels, trails, outlines, wireframe and shadow flags are recorded.
6	Motion	Rules become time blocks with start and end times. Each property is compiled into a track item. Sounds are pulled out into timed events. Look-only rules (opacity, colour, luminosity) are kept apart so they never compete with a joint.
7	Sources	Every cluster gets exactly one source of motion: a motion rule, a joint, or a gear mesh. Two sources for one cluster is error E0400, the most common modelling mistake, reported with both culprits named.
8	Spin propagation	Starting from driven parts whose axis lines up with a gear, spin flows breadth-first along mesh, couple and belt edges, multiplying ratios, so a whole gear train follows one rotate line.
9	Loops	If there are pins, the linkage solver prepares the loops and samples them through the whole animation (section 7).
10	Probe	The model is solved at the start, middle and end, so problems that only appear at run time surface as diagnostics before you press play.

SECTION 6

Solving a moment in time

`solve(model, t)` returns a world matrix for every part. It walks clusters recursively and memoises each one, so a part's matrix is computed once per call however many parts depend on it.

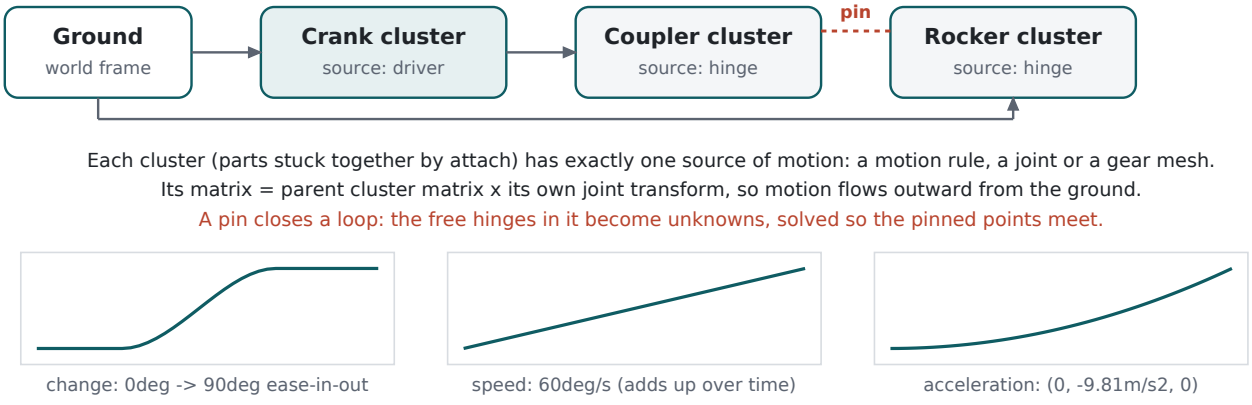


Figure 3: clusters and sources (top), and the three ways a motion value behaves over time (bottom).

Three kinds of source

- **Joint**: the parent cluster's matrix times a rotation about the hinge line (or a translation along the slider), by the joint value at t. That value comes from the joint's motion rule, from rolling contact (distance travelled divided by radius), or from the loop solver when the joint is free.
- **Driver** (a motion rule on a free part): rotations about axes or part lines, orbits around another part that follow it as it moves, offsets and velocities, travel along a Curve, and pinned coordinates, composed in a fixed order.
- **Mesh**: a rotation about the gear's own axis by the driving rotation times the accumulated ratio, with the sign flipping at each external mesh.

One evaluator for every value

All motion values go through one function, `track(items, t)`, with rules that mirror how people describe motion:

Value	Example	Behaviour at time t
Position or angle	`rotate: 90deg;`	Holds from its block start. The latest block that has started wins.
Change	`0deg -> 90deg ease-in-out`	Interpolates across its block with a CSS easing curve, then holds the end value.
Speed	`60deg/s`	Accumulates while its block runs; what it moved stays moved. Speeds from several blocks add.
Change of speed	`(0,0,0) -> (600mm/s,0,0)`	Integrates the eased speed exactly (Simpson's rule on the easing curve).
Acceleration	`(0, -9.81m/s2, 0)`	Adds a*tau^2/2 within its block.
Function of t	`35deg * sin(t * 3.14)`	Evaluated directly at t.

Colour changes use the same block rules with their own interpolation in RGB and alpha. The camera has its own evaluator (`cameraAt`) for follow, orbit, look-at, zoom and moves.

SECTION 7

Closed loops: the linkage solver

A four-bar linkage cannot be solved by walking outward from the ground: the coupler and rocker must meet. Orrery treats every unpowered hinge or slider on the chain from a pinned part back to the ground as an unknown, and every pin as three equations (the two pinned points must coincide; for an axle pin, only perpendicular to its axis).

- **Grouping.** Pins that share a free joint are grouped with a union-find; independent groups (the eight cylinders of the V8) are solved side by side, and a single perturbed solve yields a Jacobian column for every group at once.
- **Damped Newton (Levenberg-Marquardt).** Each group takes a step from the normal equations with an adaptive damping factor: accepted steps reduce damping, rejected ones increase it. Converged when the gap is under 0.00001 mm.
- **Sampling.** The loop is solved once at t = 0, then at 60 samples per second through the animation, each starting from a linear extrapolation of the two previous answers. That keeps the linkage on the same branch (it never flips into its mirror image) and makes later look-ups nearly free.
- **Look-up.** At any t the nearest sample is taken and refined with a few Newton steps, so playback at any frame rate is exact.
- **Honest failure.** If the pins cannot meet at the start, E0352 gives the best remaining gap in millimetres; if the drive asks for an impossible position later, E0353 names the moment the linkage locks.

Why sample instead of solving each frame from scratch?

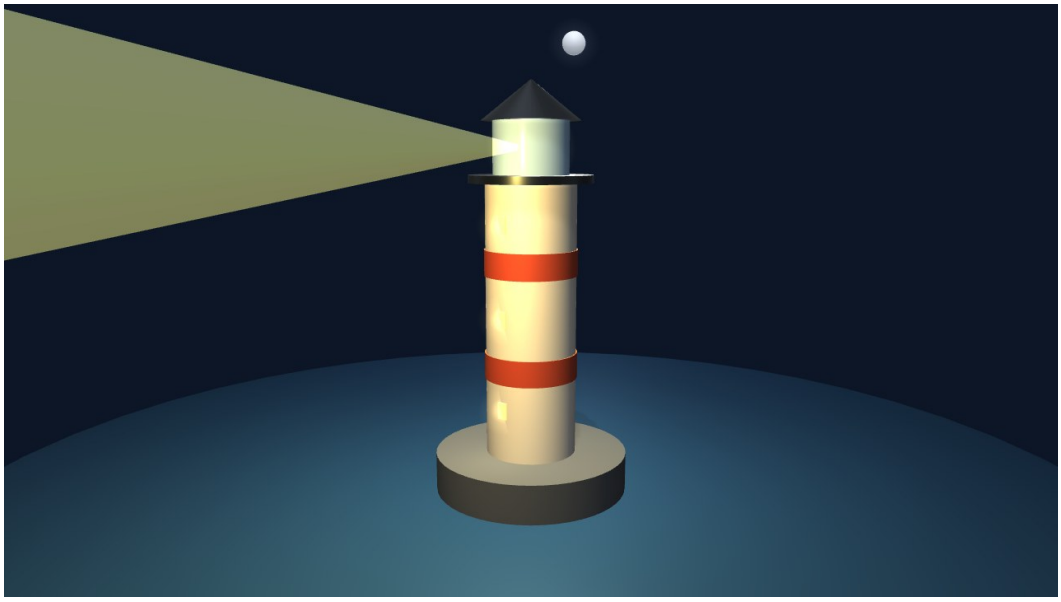
Continuity. Many linkages have two valid configurations at every crank angle. Solving each frame independently can jump between them; marching forward from the previous answer follows the real mechanism, exactly as a physical linkage would move.

SECTION 8

Drawing it: the preview renderer

The studio installs the compiled model into a three.js scene. Part matrices are written directly each frame (automatic matrix updates are switched off), so drawing costs one solve plus one render.

- **Lighting** is a fixed balance (hemisphere 0.4, fill 0.16, key 0.82, point 1.1) chosen so colours render as written. The first directional light casts soft shadows onto the parts and onto an invisible shadow catcher under the model.
- **Materials** are physically based (metalness and roughness), with a small generated studio environment for reflections on metals.
- **Luminosity** sets emissive colour, adds a soft additive halo to solid parts and gives the first eight luminous parts a real point light in their colour; see-through luminous parts blend additively, like a beam of light.
- **Per frame:** solve, then apply live style (opacity, colour and luminosity tracks), update trails (prebuilt ribbons with a moving draw range), move labels above their parts, update guides (joint axes, gear lines) and render.
- **Two cameras:** the Observer (the scene's own camera, animated) and Free (orbit controls), with a reset that returns Free to the Observer's pose.
- **Click to source:** a ray cast from a click finds the part and jumps the editor to its declaration.



The Lighthouse example: a luminous lamp and an additive beam turning at 30 degrees a second, windows whose colour and luminosity change over time, and a Moon that lights the scene softly.

SECTION 9

Errors, warnings and the overlap check

Every problem is a `Diag` object: a stable code, the file, line and column, a plain message and a concrete fix. There are 124 codes (98 errors, 26 warnings), grouped by area, and their meanings are frozen for language 1 so tools and AI agents can rely on them.

Range	Area	Typical example
`E01xx`	Syntax, let, define, repeat	E0103 Unexpected character; E0120 a let defined twice
`E02xx`	Parts and shapes	Box needs size; fewer than 6 gear teeth
`E03xx`	Names, anchors, values	E0320 there is no entity called "Rokcer"
`E04xx`	Joints and sources	E0400 two sources try to move one part
`E035x`	Loops	E0353 the linkage locks at 1.20 s
`E05xx, W05xx`	Motion and sound	E0513 a hinged part takes no "about"; W0560 overlap
`E06xx, W06xx`	Style	E0600 unknown colour; W0610 more than eight luminous parts

The overlap check

Kinematics does not prevent parts from passing through each other, so Orrery warns instead (W0560). The check runs a moment after each edit, within a time budget so typing stays smooth:

- Parts connected by joints, attachments, gears or pins, and parts one connection apart, are exempt, as are invisible parts, lines and parts marked `overlap: allow`. Pairs where neither part ever moves are skipped: they are composition, not collision.
- The animation is sampled at up to 24 moments. A pair is tested only when their bounding boxes meet; then vertices of one, pulled slightly inward, are tested for being inside the other with a ray-parity test (Moller-Trumbore triangle intersection).
- The warning names both parts and the first moment it happens.

SECTION 10

Sound

Orrery has no audio files. Each of its 34 sounds, in six groups (clock and mechanism, machines and steam, impacts, movement, signals, ambience), is a recipe: oscillators, filtered noise and envelopes driven by a seeded pseudo-random generator, so a sound is identical every time and free of copyright.

- **Declare and play:** `Knock::Sound { kind: knock; pitch: 0.8; }`, then in motion `sound: Knock at end;`, `at 2s` or `every 1s 60%`. Events are expanded at compile time into a sorted list (capped at 4,000).
- **Mixing:** events are rendered and summed into a stereo buffer, panned by where the part is on screen, and soft-limited with tanh so many sounds never clip.
- **Playback** uses Web Audio, synchronised with the timeline and scrubbing; **export** encodes the same mix as AAC (or Opus) into MP4 or WebM; the CLI writes WAV without a browser.

SECTION 11

Exports, sharing and the web app

Output	How it is made
MP4 (H.264) / WebM (VP9)	Frames rendered offscreen at the chosen size and rate, encoded with the browser's WebCodecs VideoEncoder and packaged by mp4-muxer or webm-muxer, with the sound track as AAC or Opus. Up to 1080p and 60 fps, in five aspect shapes.
GIF	Orrery's own encoder: palette quantisation and LZW compression, transparency supported.
PNG frames	Every frame as a numbered PNG in a .zip, with full alpha, for video editors.
3D player	One .html file containing the engine (three.js inlined), the project and the viewer, so anyone can scrub and orbit it offline.
Project files	.orrery (single file), .zip, three files, or JSON for tools.
Share link	The project text compressed with deflate (CompressionStream) and base64url-encoded after #p= in the address. Opening it gives the viewer a copy in its own tab; nothing is uploaded and no account exists.
Web app	A manifest and service worker make the studio installable; pages are fetched network-first and the bundled libraries cache-first, so it works offline once opened.
Offline studio	One HTML file (about 1.8 MB) with every library inside, which runs from disk with no internet.

SECTION 12

Tools around the language

Command	Does
`orrery check`	Compiles and lists problems (text or JSON); exit code 1 on errors. Includes the overlap check.
`orrery sample`	Solves the motion and prints positions, rotations, anchor points and the camera at chosen times, as JSON.
`orrery render`	Drives the studio in a headless browser to write MP4, WebM, GIF, PNG or frames; WAV needs no browser.
`orrery test`	Compares every example with saved positions at seven moments; fails if a part moves differently or stops moving.
`orrery format`	Beautifies a project (prints, or rewrites with --write).
`orrery eval`	Scores AI answers against 12 tasks with numeric checks.
`orrery prompt / mcp`	Prints the syntax prompt for any chat assistant, or runs a Model Context Protocol server exposing check, sample and render to agents.

In the studio, **Copy state** writes, as text, where every part is at the current moment (position, rotation, bounding box, joint values, nearby sounds), plus the current problems. Pasted into a chat, it lets an assistant see exactly what went wrong.

SECTION 13

Dependencies

Orrery keeps its dependencies few, permissively licensed and bundled. Nothing is fetched at run time except fonts (with system fallbacks) and, only if a bundled file is missing, a public CDN copy.

Dependency	Version	Licence	Used for	Needed
three.js	r128	MIT	Geometry, maths, WebGL rendering (core and studio)	Always
OrbitControls	r128	MIT	The Free camera	Studio
GLTFLoader	r128	MIT	Importing 3D models	On demand
CodeMirror 6	6.x bundle	MIT	The code editor (a plain editor is the fallback)	Studio
mp4-muxer / webm-muxer	5.2.2 / 5.1.4	MIT	Packaging video and audio	Video export
Google Fonts	n/a	OFL	Interface and code fonts	Optional
Node.js	18+ (22.13+ server)	MIT	CLI; optional server with built-in SQLite	CLI / server
three (npm)	0.128.0	MIT	The core in Node	CLI
Playwright or Chrome	any recent	Apache-2.0	Headless rendering from the CLI	render only

Browser features used

Feature	For	If missing
WebGL	All drawing	Nothing can be shown; a clear message explains why
WebCodecs	MP4 and WebM export	GIF and PNG frames still work
Web Audio	Sound playback and preview	Silent preview; exports unaffected
CompressionStream	Share links, zip reading	Share links unavailable
File System Access	Save in place	Downloads instead
Service Worker	Offline and install	Works online only; offline file still available

SECTION 14

Strengths

Strength	Why it matters
Motion from connections	Hinges, sliders, gears, belts, pins and rolling contact are part of the language, and closed loops are solved. One motion rule can drive a whole V8.
Errors that teach	124 stable codes, each with a fix, located to the line in whichever view you use; runtime problems such as a locking linkage are found before playback.
Built for AI	Short syntax, a prompt, an MCP server, a state readout and an evaluation set: an assistant can write, check, measure and correct a mechanism.
Zero friction	No install, no account, no server; works offline, installs as an app, shares as a link.
One truth	The CLI runs the exact compiler the browser runs, read from the same file.
Rich output	Video with generated sound, GIF, transparent frames, an interactive player.
Open	Apache-2.0, permissive dependencies, readable single-file source, static hosting.
Tested motion	Every example's positions are pinned by snapshots, and CI fails if anything moves differently.

SECTION 15

Weaknesses and limits

A fair account of what Orrery does not do, or does not yet do well.

Limit	Detail
Kinematics only	No mass, forces, friction or collision response. Motion is described, not simulated; a ball does not bounce unless you write the bounce.
Loop scope	Pins with hinges and sliders are solved; ball joints, cams and gears inside a loop are not. Very large loop systems are sampled at 60 Hz and can be slow to compile.
Overlaps are sampled	Up to 24 moments: a very fast pass-through between samples can be missed.
Rendering ceiling	Clean, not cinematic: no image textures, no post-processing, eight real lights at most. three.js is pinned at r128 to keep every project looking the same.
Single-file source	About 7,500 lines in one HTML file: simple to ship, harder for new contributors to navigate.
No community yet	One maintainer, no external tutorials or track record. This is the largest risk for anyone adopting it today.
Browser coverage	Video export needs WebCodecs (best in Chromium browsers); editing on phones is possible but cramped.
Sharing limits	Share links grow with the project and some chat apps cut long links; imported 3D model files are not included. There is no real-time collaboration.
2D and text	Typography and 2D motion graphics are basic compared with tools built for them.

SECTION 16

Quality: how it is tested

- **Motion snapshots** (tests/motion): positions and rotations of every part of all 24 examples at seven moments, compared within 0.5 mm and 0.002 in rotation; a check that something moves at all.
- **Every example compiles without warnings**, in both the three-file and single-file forms.
- **Beautify is lossless**: each example is formatted, recompiled and compared.
- **AI evaluation set**: 12 tasks, 25 numeric checks, all passing with the reference answers.
- **Browser suites**: headless Chrome drives the studio for editing, hints, the shape and sound pickers, exports with transparency, the player, share links, the offline file, installation files and the guide.
- **Server tests**: 35 checks on a throwaway database.
- **Release check**: a script lists placeholders and version mismatches before publishing.

SECTION 17

Key numbers and glossary

Measure	Value
Shape types	38 (3D parts, 2D shapes, curves, groups, meshes, sounds)
Built-in sounds	34 in 6 groups
Diagnostic codes	124 (98 errors, 26 warnings)
Examples	24, each with saved motion snapshots
Loop sampling	60 samples per second; convergence to 0.00001 mm
Overlap sampling	Up to 24 moments per check
Longest animation	120 s
Video export	Up to 1080p, 60 fps, with sound
Studio file	about 588 KB; offline copy about 1.8 MB

Term	Meaning
Anchor	A named point or line on a part (face.top, end.b, axis) used to place and connect parts.
Cluster	Parts attached together that always move as one.
Source	The one thing that moves a cluster: a motion rule, a joint or a gear mesh.
Free joint	A hinge or slider without a motion rule, whose value the loop solver finds.
Pin	A loop closure: two points that must stay together.
Track	The list of timed values for one property, evaluated by track(items, t).
Look rule	A motion rule that only changes opacity, colour or luminosity.
Diag	A diagnostic: code, location, message, fix.